

Survival Model Evaluation: `chicago_licences`

Fitted with the survival-analysis-pipeline.

Source data	<code>datasets/chicago_licences.csv.gz</code>
Rows	239,721 (82.2% with the ending observed, 17.8% censored)
Start dates	2002-01-02 to 2026-07-31
Evaluation	5 expanding-window temporal folds
Source of figures	<code>reports/chicago_demo/metrics.json</code> , regenerated by the command in section 8

1. Summary

This report evaluates two survival models fitted to `chicago_licences.csv.gz`. It holds 239,721 rows, each observed from its start date. 82.2% have observed endings and 17.8% are censored, still running when observation stopped. Median observed duration, censored rows included, is 904 days. The models predict, from what was on file at the start date, how long each row survives.

The like-for-like comparison is the fold mean. Each of the 5 temporal folds fits both models on its own window, and the fold scores average. On concordance, the share of pairs a ranking orders correctly where 0.500 is a coin flip, XGBoost AFT scores 0.585 and the Cox proportional hazards baseline 0.592. The Cox baseline scores higher.

Pooled over 143,833 out-of-time test rows, the AFT model scores 0.573 (95% bootstrap interval 0.571 to 0.575). Section 4 explains how the two figures differ.

Both models are saved, and the bundle records the Cox baseline as recommended for scoring new rows.

Most of the pooled figure reflects a row's `license_description` group. Ranking rows by their group's average prediction alone scores 0.613, and comparing only rows inside the same group scores 0.510, close to the coin flip. Section 4 gives the decomposition.

Both models' probabilities lose to a no-skill forecast at 365 days. The limitations section says what remains usable.

This dataset has no known generating process, so no computable ceiling bounds these scores, and feature attributions cannot be checked against a true mechanism.

2. Data

Durations are measured in days. These columns hold numeric codes rather than quantities and were forced to categorical: ward, community_area, police_district, zip_code.

Left truncation, where a row was already running when the source's records begin, is a data fault. Its recorded start is not its true start and nothing marks it. This pipeline has no delayed-entry handling, so those rows must be excluded during preparation. Whether they were is recorded with the dataset.

From the run's notes, written by the analyst.

For this dataset the preparation step did perform left-truncation exclusion. The portal's history is complete only starting from 2002, so a licence already running then shows its first post-2002 renewal as its recorded start, not its true start, and nothing in the data labels these rows. Keeping only licences whose earliest transaction is a genuine first issue removed 79,690 of them, 23 percent of the licences downloaded from the portal. What exposed them was a spike of 61,351 supposed starts dated 2002, against roughly 14,000 usual first issues in a normal year. No licence in this file starts before 2002. The rule and the counts are recorded in `scripts/run_prepare_chicago.py`.

Licence types that are short or event-scoped by construction are excluded as well. This includes the Special Event, Pop-Up, and Itinerant variants. That removed 23,042 licences across 12 types. These permits are intended to be temporary. Their lack of survival is obvious and expected and they do not contain useful general information about which types of business have a greater chance to survive or not. Training on those rows teaches the model to recognize short-term permit types, not to generalize.

Licence terms are visible in the curves below. The vertical drop at about two years, sharpest in the Peddler License curve, and the smaller steps at term multiples in the others, are terms expiring. A business that does not renew exits at a term boundary, so endings cluster there.

Figure 1 shows Kaplan-Meier survival curves by `license_description`, the coarsest structure in the outcome before any model is fitted.

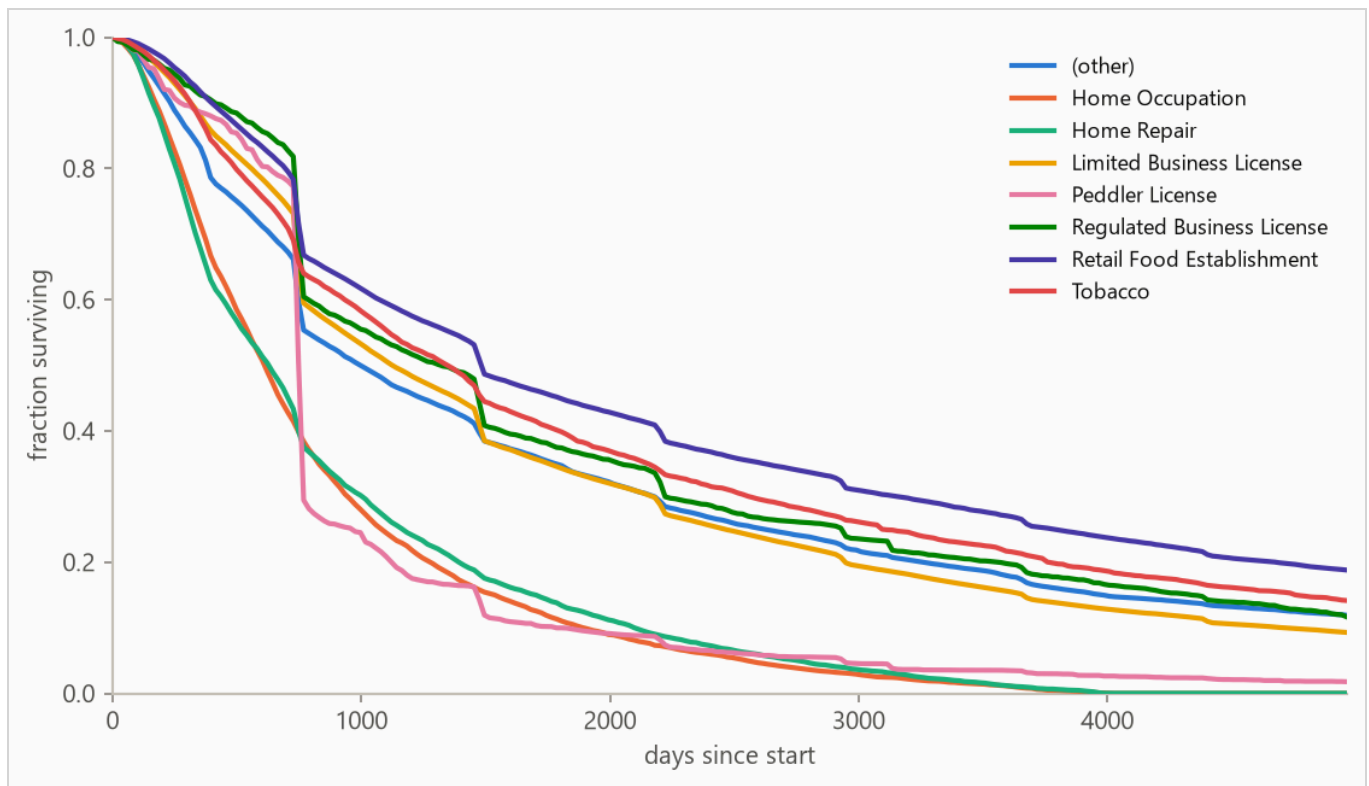


Figure 1. Kaplan-Meier survival curves by `license_description`: the fraction of each group still running at each age, with censored rows counted for as long as they were observed. Groups beyond the seven most frequent, where present, are collapsed into (other) for this plot only.

3. Method

3.1 Model class

The pipeline fits two models on every run. The first is an accelerated failure time (AFT) model, XGBoost with its `survival:aft` objective, built for durations with incomplete observations. An observed ending enters as $[t, t]$ and a censored row as $[t, \text{infinity})$, so every row contributes. It predicts each row's median survival time in days, and a log-normal curve of fitted, shared width around that median gives the probability of surviving any horizon. The second is a Cox proportional hazards baseline, the standard linear survival model, fitted with lifelines' `CoxPHFitter` on the same features. It answers whether the boosted model was necessary. The run's recommended model is whichever scores the higher fold-mean concordance.

The two differ in what they assume. The AFT model assumes every row's survival curve has the same shape and only slides it earlier or later. The Cox model makes no assumption about that shape at all, reading the curve from the data by counting who was still running at each age. In exchange it assumes each feature multiplies risk by the same factor at every age, which this report does not test. Which set of assumptions suits a dataset cannot be known in advance, which is why both are fitted and scored.

Numeric features pass through, missing values included (the Cox baseline gets train-window median imputation). Text columns are one-hot encoded with the vocabulary refit per training window, so a fold's features reflect only what was on file by its split date.

3.2 Temporal validation and label re-censoring

Evaluation uses 5 expanding-window folds ordered by start date: the earliest 40% of rows is burn-in, and each fold trains on every row started before its split date and tests on the next block (sizes in Table 2).

Training labels are re-censored at each split date. A row started long before a split may have died after it, and its label contains that future, so every post-split death is rewritten as a censoring at the split. Omitting this raises scores by importing the future, which makes it the most consequential detail in the pipeline. It is a standalone function (`temporal_folds.recensor`) with dedicated tests.

3.3 Selection and calibration

The boosted model's hyperparameters, the settings chosen rather than learned, are selected once on the first fold's training window by an inner temporal split, the same past-then-future cut made inside that window. Concordance grades only whether rows are ordered correctly, and a model can order them well while drawing survival curves far too wide or too narrow. So selection is scored on held-out censored log-likelihood instead, which grades the whole predicted distribution against what was observed.

The predictive scale is the width of the boosted model's curve, one number shared by every row. It carries no units. A larger scale spreads the model's probability across a wider range of possible lifetimes, and a smaller one concentrates it. It is measured on the most recent stretch of the training window by a model fitted only to take this measurement, then carried to the boosted model refitted on the full window. Training assumed 1.2 and the measurement found 0.88, and every boosted-model probability in this report uses the measured value.

The methodology is validated separately against synthetic data with known ground truth.

4. Results

4.1 Discrimination

Table 1. Concordance index by model, pooled over 143,833 test rows with 95% percentile bootstrap intervals over 500 resamples. Higher is better, and 0.500 is a coin flip. Fold-mean rows score each of the 5 folds separately and average. The interval resamples test rows with the fitted models held fixed, so it measures scoring precision, not stability across history. The fold spread is the guide to that.

METHOD	CONCORDANCE (HARRELL'S C)	95% INTERVAL
XGBoost AFT (pooled)	0.573	0.571 to 0.575
XGBoost AFT (fold mean)	0.585	not resampled
Cox proportional hazards (fold mean)	0.592	not resampled

Each fold fits its own models, and a Cox risk score has meaning only relative to the other rows in that same fit, so Cox scores from different folds cannot go in one list. The two models are therefore compared on fold means, each fold scored on its own and the 5 scores averaged.

The pooled row answers a different question. It scores all test rows in one list, which is enough data to attach an uncertainty range, and that range is the interval beside it. It is not the number for comparing models, because its rows were scored by 5 different fitted models. On this run it reads conservative next to the fold mean.

The boosted model's weakest window is fold 3, starting 2013-11-27, at 0.554. Any established cause is a dataset finding, and the run's notes are where it would appear.

The pooled concordance decomposes by `license_description`. Ranking rows by their group's average gives every row in a group the same score, so comparisons only ever run between groups, and those come out right 61.3% of the time. The boosted model scores each row individually instead, which orders rows inside a group and can also move one past rows in other groups. Inside a group it is right 51.0% of the time, pair-weighted across 65 qualifying groups.

Figure 2 plots the per-fold concordance from Table 2.

Table 2. Per-fold results. Censoring is the share of each fold's test rows whose ending was not observed.

FOLD	SPLIT DATE	TRAIN N	TEST N	CENSORED	AFT	COX
1	2009-05-18	95,870	28,767	7%	0.600	0.595
2	2012-03-27	124,601	28,767	14%	0.558	0.558
3	2013-11-27	153,398	28,767	18%	0.554	0.599
4	2017-08-07	182,164	28,766	29%	0.601	0.592
5	2021-11-02	210,947	28,766	67%	0.610	0.615

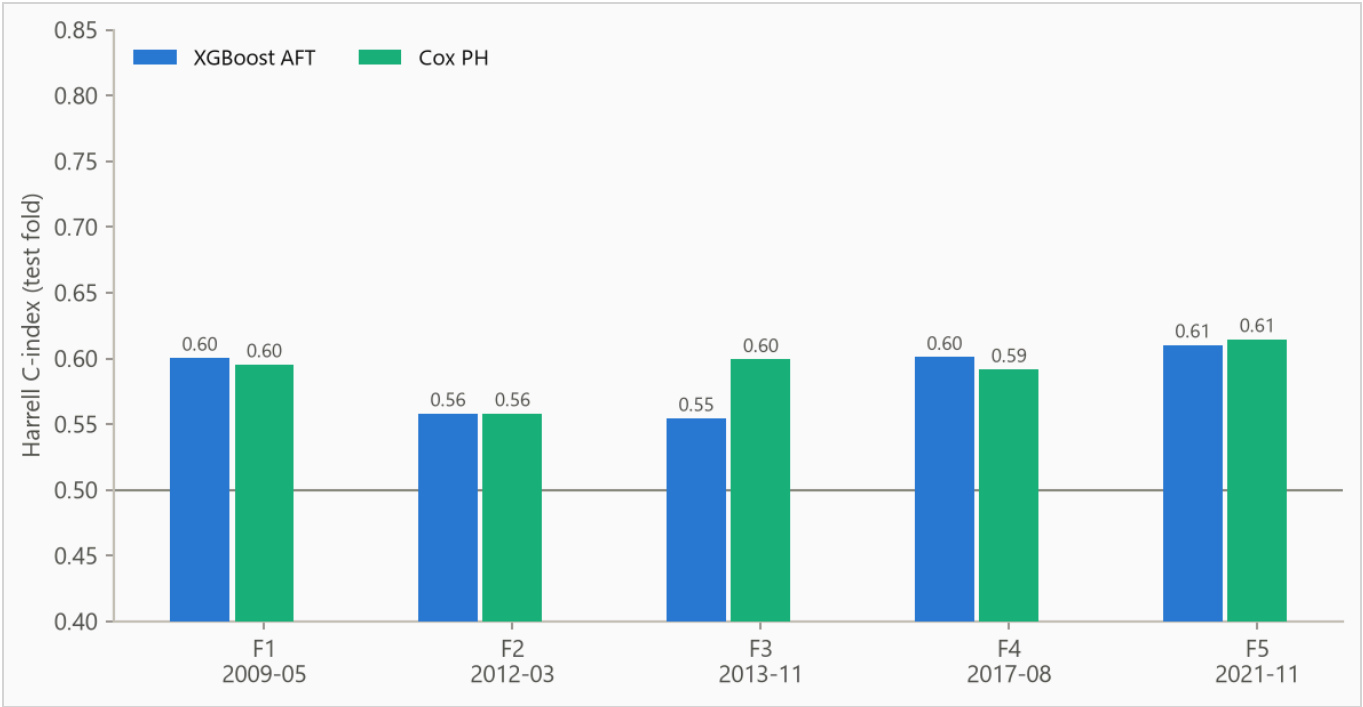


Figure 2. Concordance by fold. The boosted model spans 0.554 to 0.610. Folds differ in censoring mix, so cross-fold comparisons are indicative rather than exact.

4.2 Calibration

The Brier score, the mean squared error of a probability forecast (lower is better), is weighted by the inverse probability of censoring (IPCW) so censored rows do not bias it. The no-skill reference assigns every row the population's Kaplan-Meier marginal, the fraction surviving at each age with censored rows counted while observed.

Table 3. IPCW Brier score by horizon. Lower is better.

HORIZON	AFT	COX	NO-SKILL MARGINAL
365 days	0.097	0.088	0.081
1095 days	0.239	0.231	0.250
1825 days	0.217	0.212	0.227

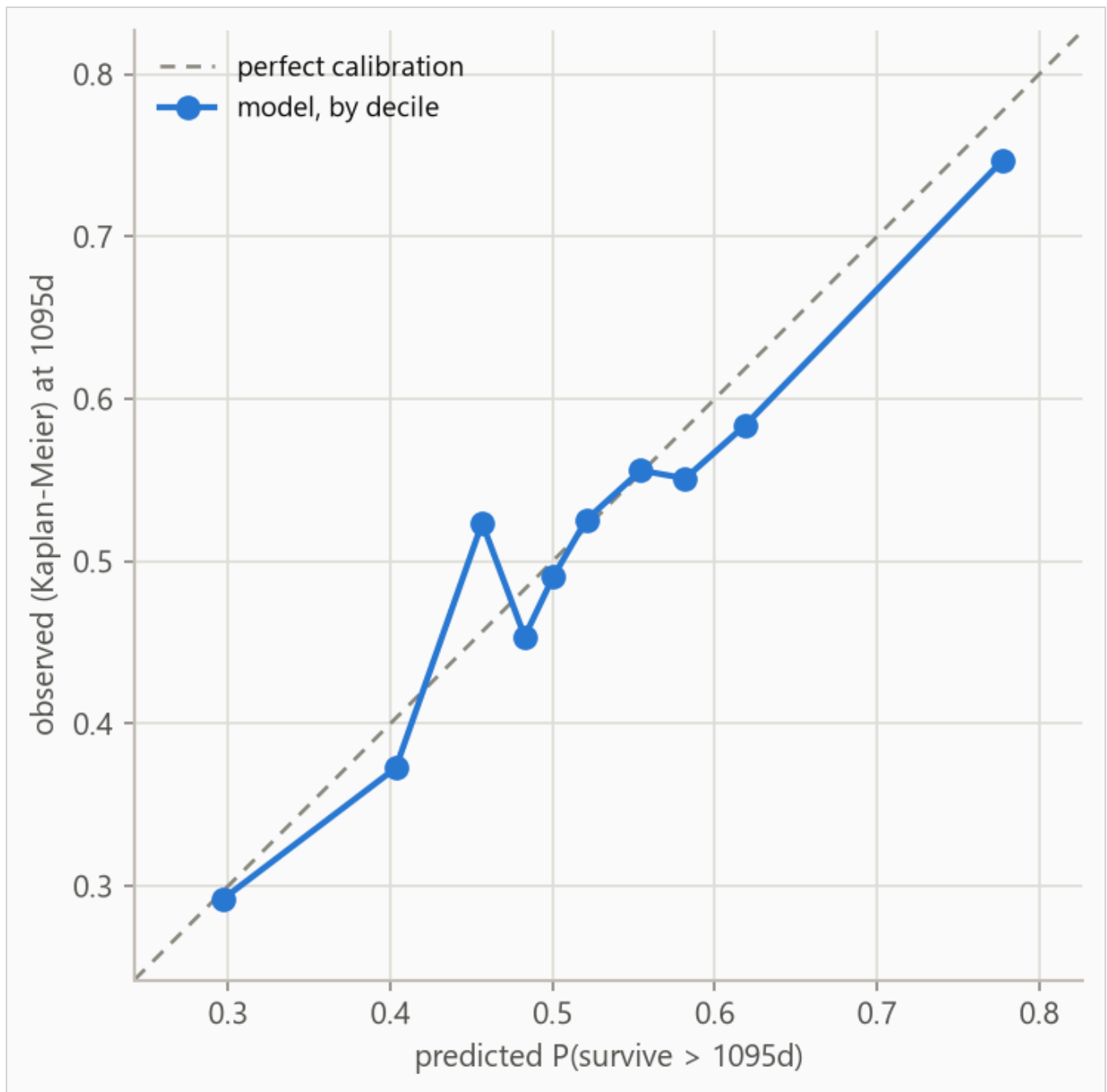


Figure 3. Predicted against observed survival at 1095 days for the boosted AFT model, by predicted decile. Observed frequencies are Kaplan-Meier estimates within each bin, so censored rows contribute correctly. The largest deviation is 0.067 in decile 3.

Table 4. Decile calibration at 1095 days for the boosted AFT model, the values plotted in Figure 3. Deciles are cut on predicted value, so tied predictions make the counts uneven. The observed value in each is a Kaplan-Meier estimate. When a decile's last observed row falls short of the 1095-day horizon the estimate carries its last value forward, so in small or heavily censored deciles the observed column is softer than its three decimals look.

DECILE	N	PREDICTED	OBSERVED (KM)	DEVIATION
1	14492	0.297	0.292	-0.005
2	14933	0.403	0.373	-0.031
3	13805	0.457	0.524	+0.067
4	15540	0.483	0.454	-0.029
5	13533	0.501	0.491	-0.010
6	14076	0.521	0.526	+0.005
7	14318	0.554	0.556	+0.002
8	15061	0.582	0.551	-0.031
9	13874	0.619	0.584	-0.035
10	14201	0.777	0.747	-0.031

5. What the boosted model uses

Feature attributions (SHAP values, for SHapley Additive exPlanations) are computed for the boosted model on the log scale of survival time. A +0.3 attribution multiplies predicted survival by about 1.35, and negative values shorten it.

Attributions are descriptive and in-sample, computed on the final boosted model's own training rows. Correlated features split credit by the model's internal choices as much as by the data, so directions are more trustworthy than magnitudes.

Figure 4 ranks features, Table 5 lists the top eight, Figure 5 shows the per-row spread, and Figure 6 traces attribution against value.

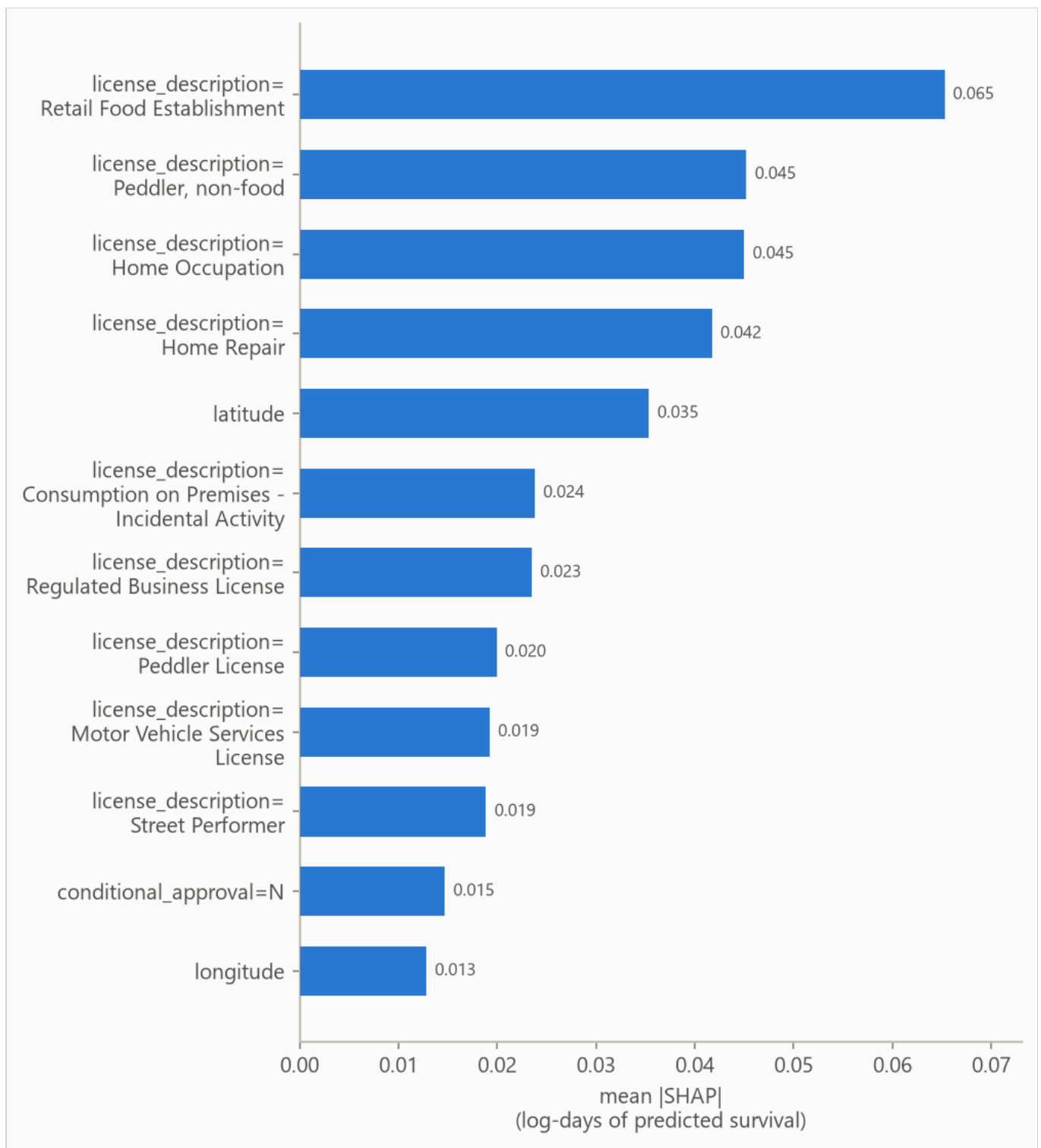


Figure 4. Mean absolute attribution by feature.

Table 5. Top eight features by mean absolute attribution.

FEATURE	MEAN ATTRIBUTION
license_description=Retail Food Establishment	0.065
license_description=Peddler, non-food	0.045
license_description=Home Occupation	0.045
license_description=Home Repair	0.042
latitude	0.035
license_description=Consumption on Premises - Incidental Activity	0.024
license_description=Regulated Business License	0.023
license_description=Peddler License	0.020

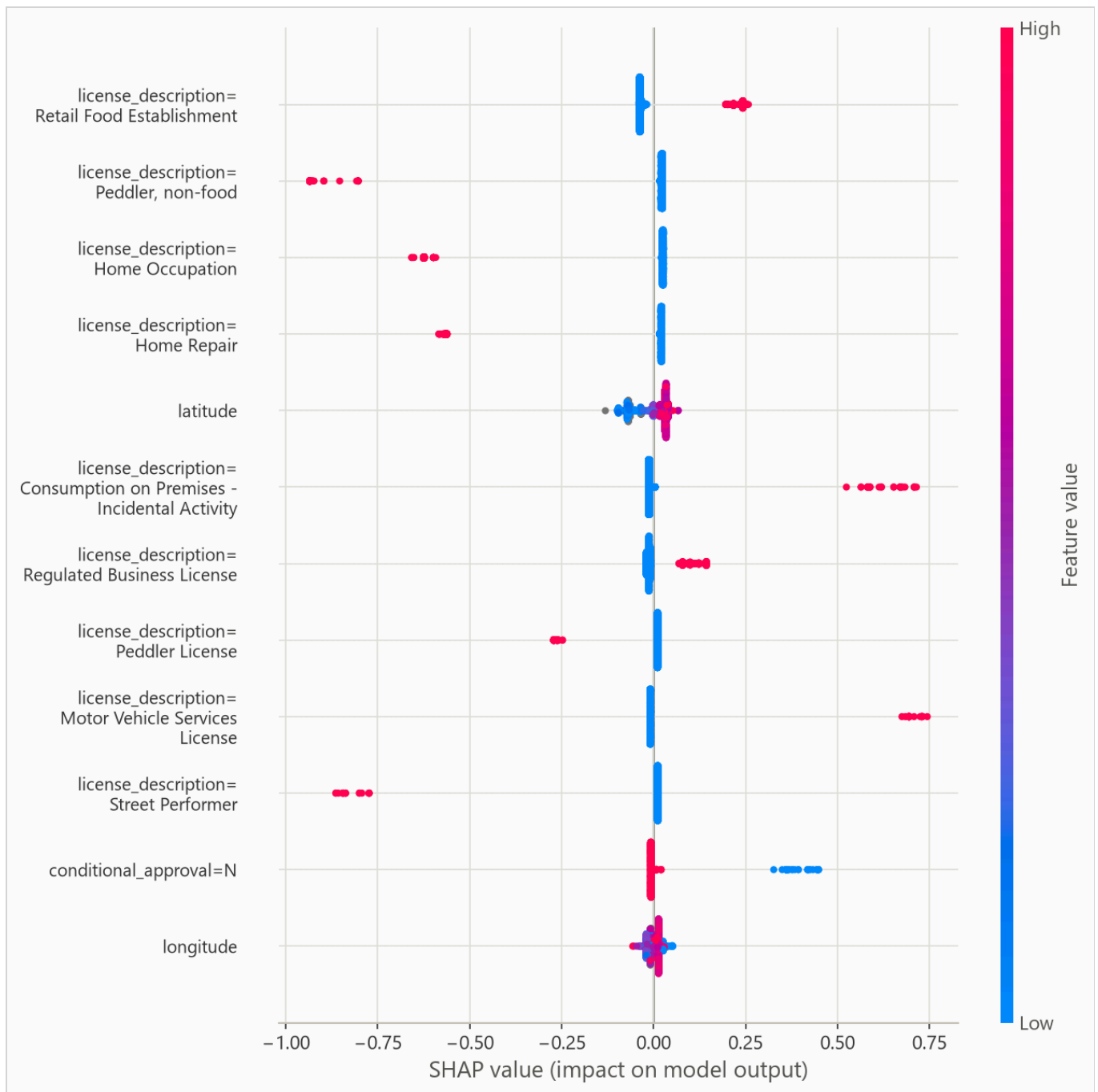


Figure 5. Per-row attributions across the explanation sample. For a yes/no feature, such as a one-hot category flag, the high (red) end simply means the row is in that category.

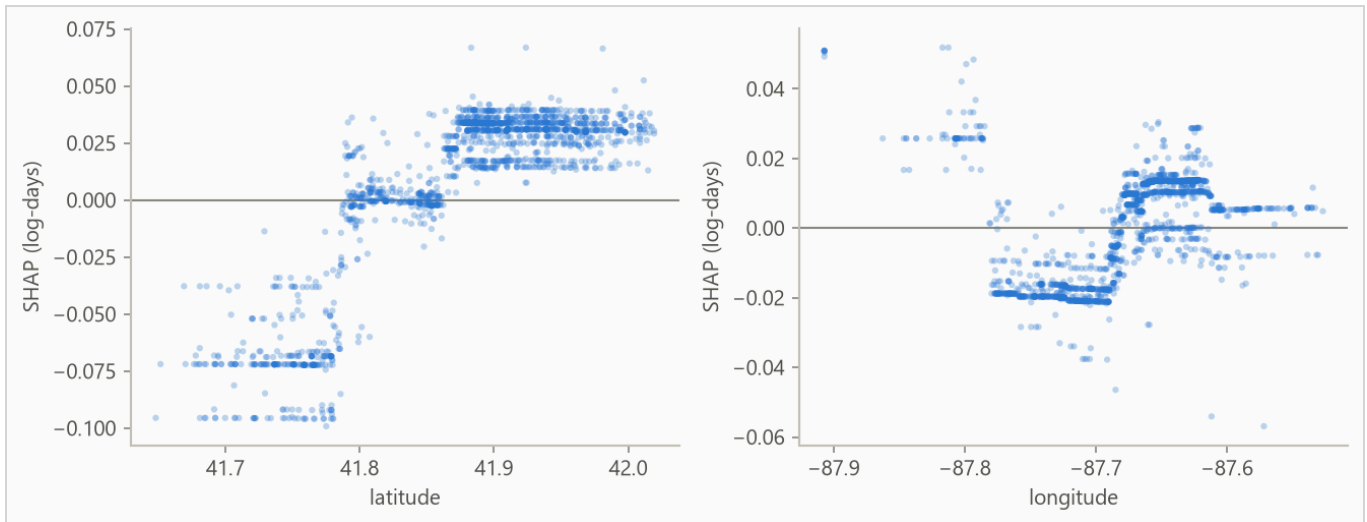


Figure 6. Attribution against feature value for the strongest numeric features. Category flags carry no shape and stay in the ranking above. A run short on numeric features fills the grid with flags instead.

On this data there is no known mechanism to check the ranking against, so read it as "what this model leaned on", not as importance in the world.

6. Interpretation

From the run's notes, written by the analyst.

The comparison worth acting on is the model choice. The Cox baseline edges the boosted model on the fold mean, 0.592 to 0.585, and the honest reading is that on this problem a penalized linear model is enough. Day-one paperwork mostly identifies risky licence categories. Ranking rows by their category's mean prediction alone scores 0.613 against the model's pooled 0.573, and comparisons within a category score 0.510, barely above chance. So, for example, a Home Repair business correlates with short life, but the metadata says almost nothing about which handyman outlasts the others.

The one-year probabilities should not drive decisions. The model loses to the no-skill reference on Brier score at that horizon, 0.097 against 0.081, so treat it as an ordering tool at short horizons rather than a probability source there.

Fold 3, the weakest window in the per-fold table, drops for a measurable reason. Upon investigation it was discovered that its test block sits across a shift in the city's category mix, and the shift is administrative rather than economic. The city stopped issuing the Home Occupation and Home Repair licence types in 2012, the same year Regulated Business License first appears with a one-year spike of issues. The change is the city relabeling businesses, not the businesses themselves changing, and this event was discoverable in the data.

The two retired categories carry 11 percent of this fold's training rows and stop appearing in the test block entirely. Regulated Business License nearly triples its share. Another 2 percent of test rows fall in categories the training window never saw. Re-selecting hyperparameters on that fold's own window closes about a third of its gap to the Cox baseline on that fold. This demonstrates that a fold that drops as this one did is worth investigating against the composition of its test block before being dismissed as model instability. The measurements in this and the previous paragraph are recomputed by

```
scripts/run_fold3_investigation.py .
```

The printed bootstrap interval is narrower than the real uncertainty. Licences in the same category and the same stretch of time tend to fail together, and the interval resamples rows as if they were independent. The per-fold spread in the results section is the better guide to how the score moves across history.

7. Limitations

Absolute probabilities are not usable at 365 days. Both models lose to the no-skill forecast on the censoring-weighted Brier score there, and Table 3 grades each model separately. Ranking and probability quality are separate, so ordering rows is still supported at those horizons.

Censoring may be informative. The evaluation assumes rows stop being observed for reasons unrelated to their risk. Early exits of rows about to end anyway bias survival estimates upward.

From the run's notes, written by the analyst.

Endings near the cutoff are provisional. Where no cancellation exists, a licence's end can only be its latest recorded expiry. Inevitably, a licence whose term ran out shortly before the cutoff but renewed late cannot be told apart from a real closure. As such, it is recorded as an observed closure. Deaths dated close to the cutoff therefore include some licences that will turn out to have survived. The only way to know is to continually repull and update the dataset as time passes.

The boosted model gives every row one curve shape. The predictive scale is a single fitted number, so it shifts a row's survival curve earlier or later but never reshapes it. Per-row width is future work.

Harrell's concordance is biased under heavy censoring, and the most censored test window is 67% censored. Read Table 2 with that in mind.

8. Reproducing this run

Python 3.11 or later. From the repository root:

```
python scripts/run_fit_evaluate.py --data datasets/chicago_licences.csv.gz --name chicago_licences
python scripts/run_build_report.py --run reports/chicago_demo
```

Every number is read from the run's `metrics.json` at build time, and a figure the prose never cites fails the build. `requirements.txt` pins the exact versions the committed numbers came from.

Generated from `reports/chicago_demo/metrics.json` by `scripts/run_build_report.py --run .` 239,721 rows, 143,833 out-of-time test rows.